

アカウント情報編集機能の実装

この機能は、仕様書にある「**更新後は数秒のインターバル後に自動ログアウトする**（変更を安全に反映し、再ログインを促す）」というセキュリティ思想を伴います。

新設する必要があるコンポーネントとバックエンド機能の設計を以下のようにまとめております。

フロントエンド側の設計事項

新しく `AccountEditView.vue` というコンポーネントを作成し、ルーターに登録します。

画面の構成要素（UI）

- **現在の情報の表示**：ユーザーID（メールアドレス）
- **編集フォーム**：新しい「ユーザー名」および「新しいパスワード / 確認用パスワード」の入力欄
- **3秒カウントダウン通知ボックス**：登録画面で作成したロジックを応用し、更新成功時に「情報を更新しました。安全のため3秒後に自動ログアウトします...」と表示するエリア。

核心となるフロント処理

- 画面表示時に、ローカルストレージからログイン後の `accountId` と `userName` を復元して初期値としてセットします。
- 「変更を保存する」ボタンが押されたら、バックエンドの更新APIへデータを送信（`PUT /api/users/update`）。

- ローカルストレージのログイン情報（`localStorage.removeItem('user')`）をクリアします。
- 画面にカウントダウンを表示し、3秒後に強制的に `/login` 画面へリダイレクトさせます。

バックエンド側の設計事項

既存のユーザー管理系（`UserController.java` 周辺）に、情報更新用のエンドポイントを追加します。

核心となるバックエンド処理

- `UserController.java` に `@PostMapping("/update")` を新設。
- フロントから送られてきた `accountId` を基に、DBから既存のユーザー情報を検索。
- ユーザー名が変更されていれば上書き。
- **パスワードが入力されている場合**、既存の新規登録時と同じハッシュ化ロジック（BCrypt等）を通して安全に暗号化してからDBに保存（`userRepository.save()`）。

新たに追加・修正するファイル一覧

着手するにあたり、作成・修正が必要なファイルへの道標です。

フロントエンド (frontend/src)

- `views/AccountEditView.vue` 【新設】：アカウント編集画面本体。
- `router/index.js` 【修正】： `/account-edit` というURLで画面が開くようにルーティングを追加。
- `components/AppHeader.vue` 【修正】：ハンバーガーメニュー内に「アカウント情報編集」へのリンク（ `router.push('/account-edit')` ）を追加。

バックエンド (backend/src)

- `controller/UserController.java` 【修正】：更新受付窓口（ `@PostMapping("/update")` ）の追加。
- `service/UserService.java` 【修正】：DBの書き換えロジック（ `updateUser` メソッドなど）の追加。

今回のバックエンドの処理の流れ

コードを書き換える前に、どのような仕組みを作るのかイメージしておきましょう。

- **リクエストの受付**

フロントエンドから、`accountId`（誰の）、`userName`（新しい名前）、`password`（新しいパスワード：任意）を含んだデータを受け取ります。

- **既存ユーザーの検索**

送られてきた `accountId` をキーに、データベースから現在のユーザー情報を引っ張ってきます。

- **データのマッピング（上書き）**

- ユーザー名は無条件で新しい名前に書き換えます。
- パスワードは、「入力されている場合のみ」新しく暗号化（ハッシュ化）して上書きします（空っぽで届いた場合は、現在のパスワードを維持します）。

- **保存とレスポンス**

データベースに `save` し、フロントエンドに「更新成功」の合図を返します。

UserAccountService.java の修正（ロジックの追加）

今回の設計のポイントは、「パスワードが空で届いた場合は現在のパスワードを維持し、入力されている場合のみ新しくハッシュ化して上書きする」という柔軟性を持たせる点です。

まずは、データベースの検索と更新の実務を担う `UserAccountService` に、アカウント更新用のメソッドを追加します。既存のクラスの最後の閉じ括弧 `}` の手前に、以下のメソッドを書き足してください。

```
/**
 * アカウント情報の更新処理
 * @param requestData フロントから届いた更新用データ（accountId, userName, passwordを含む）
 * @return 更新後のユーザー情報
 */
public UserAccount updateUser(UserAccount requestData) {
    // 1. accountId を基に、既存のユーザーをデータベースから検索
    UserAccount existingUser = repository.findById(requestData.getAccountId())
        .orElseThrow(() -> new RuntimeException("ユーザーが見つかりません。"));

    // 2. ユーザー名の更新
    if (requestData.getUserName() != null && !requestData.getUserName().trim().isEmpty()) {
        existingUser.setUserName(requestData.getUserName().trim());
    }
}
```

```
// 3. 【重要】パスワードの更新チェック
// パスワードが入力されている (nullでも空文字でもない) 場合のみ、ハッシュ化して上書き
if (requestData.getPassword() != null && !requestData.getPassword().trim().isEmpty()) {
    String encodedPassword = passwordEncoder.encode(requestData.getPassword());
    existingUser.setPassword(encodedPassword);
}

// 4. 自己紹介 (about) など、もし他の編集可能フィールドがあれば同様にマッピング
if (requestData.getAbout() != null) {
    existingUser.setAbout(requestData.getAbout());
}

// 5. データベースへ上書き保存 (JPAの仕様により、既存レコードへのsaveはUPDATE文になります)
return repository.save(existingUser);
}
```

※ もし `repository.findById()` でコンパイルエラーが出る場合は、`UserAccountRepository` の定義に合わせて `repository.findById()` ではなく、主キー (`accountId`) で検索できるメソッド (標準の `findById` など) を利用してください。

UserAccountController.java の修正（窓口の追加）

次に、フロントエンドからのリクエスト（PUT /api/users/update）を受け付ける窓口を新設します。

UserAccountController クラスの内部に、以下のエンドポイントを追加してください。

```
/**
 * アカウント情報の更新を受け付けるエンドポイント
 * PUT http://localhost:8080/api/users/update
 */
@PutMapping("/update")
public ResponseEntity<String> updateAccount(@RequestBody UserAccount accountRequest) {
    try {
        userService.updateUser(accountRequest); // サービス層の更新処理を呼び出す

        return ResponseEntity.ok("アカウント情報を更新しました。"); // Vue側で「更新成功」をトリガーにするためのメッセージを返す
    } catch (RuntimeException e) {
        // ユーザーが見つからないなどのエラー時は 400 Bad Request
        return ResponseEntity.badRequest().body("更新に失敗しました: " + e.getMessage());
    }
}
```


バックエンドの実装のポイント

1. JPAによる安全な UPDATE

`requestData` をそのまま `save()` するのではなく、一度DBから `existingUser`（既存の全データが入った状態）を引っ張ってきてから必要な箇所だけを書き換えています。これにより、メールアドレス（`userId`）や作成日時（`createdAt`）、権限（`isAuth`）といった編集画面で触らない大切なデータが `null` で上書きされて消えてしまう事故を完全に防ぎます。**

2. パスワード変更の任意性

「ユーザー名だけ変更して、パスワードは今のままにしたい」というケースに対応するため、`requestData.getPassword()` の中身が空っぽであればスルーし、現在のハッシュ化済みパスワードをそのまま維持する安全設計にしています。

フロントエンドの実装

SpringBootの軌道に成功したら、バックエンド側は正常に機能していることを確認できます。

ここからは次のステップである**フロントエンド（Vue.js）の実装**へと移りましょう！

今回は、新しく作成するアカウント編集画面（ `AccountEditView.vue` ）のソースコードと、それをアプリ内で動かすためのルーティング等の設定手順についてをまとめました。

AccountEditView.vue の作成

他のコンポーネント用ファイルも格納されている `components/` フォルダの中に、新しく `AccountEditView.vue` というファイルを作成し、以下のコードを貼り付けてください。

```
<template>
  <div class="account-edit-container">
    <h2>アカウント情報編集</h2>

    <div v-if="countdown > 0" class="alert alert-success">
      <p>アカウント情報を更新しました。</p>
      <p>安全のため、<strong>{{ countdown }}秒後</strong>に自動ログアウトします...</p>
    </div>

    <form v-else @submit.prevent="handleUpdate" class="edit-form">
      <div class="form-group">
        <label>ユーザーID（メールアドレス）</label>
        <input type="text" :value="userId" disabled class="input-disabled" />
        <small class="form-help">※ユーザーIDは変更できません。</small>
      </div>
    </form>
  </div>
```

```
<div class="form-group">
  <label for="userName">新しいユーザー名</label>
  <input type="text" id="userName" v-model="userName" required placeholder="ユーザー名を入力" />
</div>

<div class="form-group">
  <label for="password">新しいパスワード</label>
  <input type="password" id="password" v-model="password" placeholder="変更する場合のみ入力" />
</div>

<div class="form-group">
  <label for="passwordConfirm">新しいパスワード（確認）</label>
  <input type="password" id="passwordConfirm" v-model="passwordConfirm" placeholder="もう一度入力" />
</div>

<p v-if="errorMessage" class="error-message">{{ errorMessage }}</p>

<button type="submit" :disabled="isSubmitting" class="btn-submit">
  {{ isSubmitting ? '更新中...' : '変更を保存する' }}
</button>
</form>
</div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import apiClient from '../api'; // ※お使いの共通APIクライアントのパスに合わせて調整してください

const router = useRouter();

// フォーム用の状態管理
const accountId = ref('');
const userId = ref('');
const userName = ref('');
const password = ref('');
const passwordConfirm = ref('');

const errorMessage = ref('');
const isSubmitting = ref(false);
const countdown = ref(0); // 自動ログアウトまでのカウントダウン秒数

// 画面表示時にローカルストレージから現在の情報を復元
onMounted(() => {
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    accountId.value = user.accountId;
    userId.value = user.userId; // エンティティ定義の userId (メールアドレス)
    userName.value = user.userName; // 現在の名前を初期値としてセット
  } else {
    // ログイン情報がなければログイン画面へ弾く
    router.push('/login');
  }
});
```

// 更新処理

```
const handleUpdate = async () => {  
  errorMessage.value = '';
```

// パスワードのバリデーション（入力されている場合のみチェック）

```
if (password.value) {  
  if (password.value !== passwordConfirm.value) {  
    errorMessage.value = 'パスワード（確認）が一致しません。';  
    return;  
  }  
  if (password.value.length < 4) { // 必要に応じて桁数は調整してください  
    errorMessage.value = 'パスワードは4文字以上で入力してください。';  
    return;  
  }  
}
```

```
isSubmitting.value = true;
```

```
try {
  // 先ほどJava側で作成した PUT /api/users/update を叩く
  await apiClient.put('/api/users/update', {
    accountId: accountId.value,
    userName: userName.value,
    password: password.value || null // 空白ならnullを送り、Java側で維持させる
  });

  // 【成功時の処理】
  isSubmitting.value = false;
  countdown.value = 3; // 3秒のカウントダウンを開始

  // タイマーを回す（1秒ごとにカウントを減らす）
  const timer = setInterval(() => {
    countdown.value--;
    if (countdown.value === 0) {
      clearInterval(timer);

      // ローカルストレージを綺麗に消去（ログアウト処理）
      localStorage.removeItem('user');

      // ログイン画面へ強制リダイレクト
      router.push('/login');
    }
  }, 1000);
} catch (error) {
  isSubmitting.value = false;
  if (error.response && error.response.data) {
    errorMessage.value = error.response.data;
  } else {
    errorMessage.value = '通信エラーが発生しました。時間をおいて再度お試しください。';
  }
}
};
</script>
```

```
<style scoped>
/* シンプルでモダンなフォームデザイン（必要に応じて既存の共通スタイルと合わせてください） */
.account-edit-container {
  max-width: 500px;
  margin: 40px auto;
  padding: 30px;
  background: #ffffff;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);
}
h2 {
  margin-bottom: 20px;
  color: #333;
  text-align: center;
}
.form-group {
  margin-bottom: 20px;
}
label {
  display: block;
  margin-bottom: 6px;
  font-weight: bold;
  color: #495057;
}
input {
  width: 100%;
  padding: 10px;
  border: 1px solid #ced4da;
  border-radius: 4px;
  box-sizing: border-box;
}
.input-disabled {
  background-color: #e9ecef;
  color: #6c757d;
  cursor: not-allowed;
}
```



```
.form-help {
  color: #6c757d;
  font-size: 0.85rem;
}
.error-message {
  color: #dc3545;
  font-weight: 500;
  margin-bottom: 15px;
}
.btn-submit {
  width: 100%;
  padding: 12px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 4px;
  font-size: 1rem;
  font-weight: bold;
  cursor: pointer;
  transition: background-color 0.2s;
}
.btn-submit:hover {
  background-color: #0056b3;
}
.btn-submit:disabled {
  background-color: #6c757d;
  cursor: not-allowed;
}
.alert-success {
  background-color: #d4edda;
  color: #155724;
  padding: 20px;
  border-radius: 4px;
  border-left: 5px solid #28a745;
  text-align: center;
}
</style>
```

ルーティングへの登録 (router/index.js など)

URL (/account-edit) でアクセスできるようにルーティング設定ファイルに追加します。

```
// router/index.js の既存のルート配列 (routes) の中に以下を追加します
{
  path: '/account-edit',
  name: 'AccountEdit',
  component: () => import('../views/AccountEditView.vue') // パスは環境に合わせ指定してください
}
```

動作テストの確認ポイント

ファイルを保存したら、ブラウザで直接 URL に `/account-edit` を打ち込むか、あるいはダッシュボード等のボタンからこの画面へ遷移させてみてください。

1. **初期表示**：現在のユーザー名が最初から入力欄に入っていること、メールアドレスがグレースアウトして固定されていることを確認します。
2. **名前だけ変更テスト**：名前を少し変えて「変更を保存する」を押します。3秒のカウントダウンが表示され、ログイン画面へパッと戻れば成功です。その後、新しい名前でダッシュボードにログインできるか試します。
3. **パスワード変更テスト**：新しいパスワードを入力して保存し、自動ログアウト後、**新しいパスワード**でしかログインできなくなっていることを確認します。